

# Pemrograman untuk Ilmu Data dan Komputasi

## Pertemuan 1: Kontrak Kuliah dan Python Dasar

MA25-21008 Kelas RA dan RB

Rifky Fauzi

Triyana Muliawati, S.Si., M.Si.

Program Studi Matematika  
Institut Teknologi Sumatera  
Semester Ganjil 2026/2027

### Jadwal Mingguan

| Jenis     | Kelas | Hari  | Waktu       | Ruang    |
|-----------|-------|-------|-------------|----------|
| Kuliah    | RA    | Jumat | 10.15-11.55 | F212(80) |
| Kuliah    | RB    | Senin | 10.15-11.55 | F204(80) |
| Praktikum | RA    | Senin | 10.00-12.00 | Lab      |
| Praktikum | RB    | Jumat | 10.00-12.00 | Lab      |

Lab = Lab. Komputasi Matematika dan Statistika

## Konteks perkuliahan

- Mengapa pemrograman penting untuk Matematika.
- Computational thinking dan keterampilan abad 21.
- Mengapa Python relevan untuk data dan scientific computing.
- Kontrak kuliah, asesmen, dan project PKM.

## Python dasar

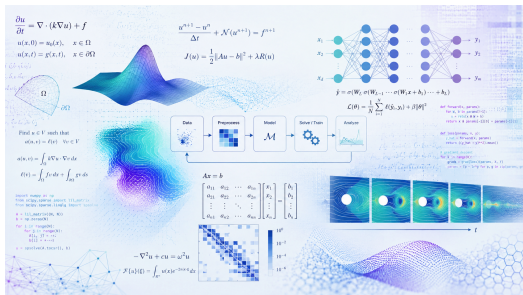
- Model mental program Python.
- Variabel, input, output.
- Tipe data `int` dan `float`.
- Aritmatika, `if`, dan perulangan.
- Tugas mandiri dan quiz e-learning.

## Buku rujukan utama

| Buku                  | Urgensi dalam MK                                                                                                 | Bagian SAP yang didukung          |
|-----------------------|------------------------------------------------------------------------------------------------------------------|-----------------------------------|
| Miller & Ranum (2013) | Problem solving, algoritma, data, variabel, control structures, functions, Big-O, recursion, searching, sorting. | Fondasi IndoMS M1-M6              |
| Pine (2019)           | Scientific computing: NumPy array, vektor, matriks, sistem linear, dan komputasi numerik.                        | Komputasi Matematika M3 dan M7    |
| McKinney (2022)       | NumPy, pandas, Jupyter, wrangling, visualisasi, workflow data, dan pengantar modeling.                           | Data analysis/data science M9-M12 |

**Keputusan:** Pine tidak diganti. Miller & Ranum memperkuat algoritma dan Big-O; McKinney menjaga arah data analysis dan data science.

# Mengapa pemrograman penting bagi mahasiswa Matematika?



## Matematika memberi struktur

- Variabel, fungsi, matriks, peluang, dan optimisasi.
- Asumsi, pembuktian, dan validasi hasil.
- Model yang menjelaskan fenomena.

## Pemrograman memberi daya eksekusi

- Menguji model pada data nyata.
- Menjalankan eksperimen berulang.
- Mengubah ide menjadi alat, simulasi, atau analisis.

# Computational thinking dan keterampilan abad 21

## Computational thinking

- Dekomposisi masalah.
- Pengenalan pola.
- Abstraksi.
- Desain algoritma.
- Debugging dan evaluasi.

## Keterampilan abad 21

- Critical thinking: menilai strategi dan hasil.
- Creativity: merancang solusi baru.
- Collaboration: membaca dan memperbaiki kode bersama.
- Communication: menjelaskan proses, output, dan keterbatasan.

Rujukan: konsep 4C dari P21; computational thinking dipopulerkan antara lain oleh Jeannette Wing.

# Student-centered learning: bukan dosen tidak mengajar

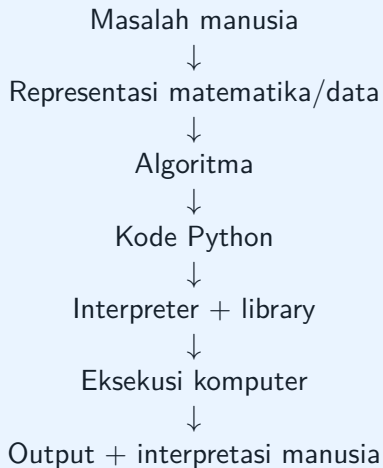
## Posisi dosen dan mahasiswa

- Dosen berperan sebagai fasilitator: memberi arah, contoh, tantangan, dan umpan balik.
- Mahasiswa aktif membaca, mencoba, salah, memperbaiki, dan menjelaskan ulang.
- Sharing sesama mahasiswa adalah bagian dari proses belajar, bukan pengganti dosen.

## Mengapa demikian?

- Pemrograman tidak cukup dipahami dengan mendengar penjelasan.
- Belajar terbaik terjadi ketika kita mencoba dan menjelaskan kepada orang lain.
- Di kelas ini, diskusi dan peer sharing adalah latihan berpikir komputasional.

## Model mental Python



## Sumber model mental

Diadaptasi dari Miller & Ranum (2013) tentang problem solving, algoritma, data, control structures, dan functions; Pine (2019) tentang scientific computing; serta McKinney (2022) tentang Python basics, NumPy, pandas, dan Jupyter.

### Tidak satu penemu tunggal

- Charles Babbage: rancangan Analytical Engine.
- Ada Lovelace: algoritma untuk mesin Babbage, sering disebut programmer pertama.
- Alan Turing: model abstrak komputasi dan batas komputabilitas.
- John von Neumann: arsitektur komputer modern dan komputasi numerik.
- Grace Hopper: compiler dan bahasa pemrograman yang lebih dekat ke manusia.

**Pesan:** teknologi komputasi berkembang sangat dekat dengan logika, aljabar, algoritma, statistik, dan pemodelan.

# Perkembangan bahasa pemrograman

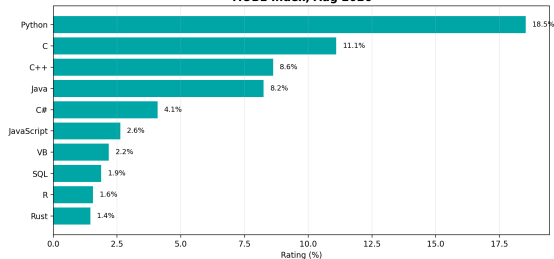
| Periode  | Bahasa/contoh                  | Makna perkembangan                                                     |
|----------|--------------------------------|------------------------------------------------------------------------|
| 1940-an  | Assembly                       | Dekat dengan instruksi mesin. Cepat, tetapi sulit dibaca.              |
| 1950-an  | FORTRAN, LISP, COBOL           | Komputasi ilmiah, simbolik, dan bisnis mulai terstruktur.              |
| 1970-an  | C, Pascal, SQL                 | Sistem operasi, struktur data, dan basis data.                         |
| 1980-an  | MATLAB, C++                    | Komputasi matriks dan pemrograman berorientasi objek.                  |
| 1990-an  | Python, R, Java, JavaScript    | Produktivitas, web, statistik, scripting, dan ekosistem library.       |
| 2010-an+ | Julia, Rust, Go, Python modern | Performa, keamanan, cloud, AI, data science, dan scientific computing. |

**Inti:**

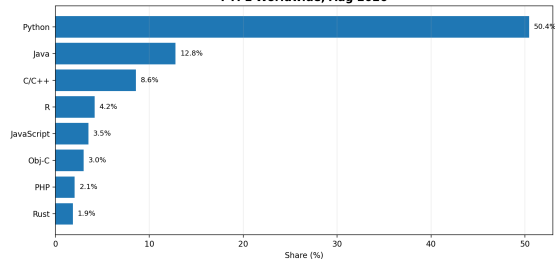
bahasa berubah, tetapi pola berpikir algoritmik tetap bertahan.

# Data terbaru: mengapa Python?

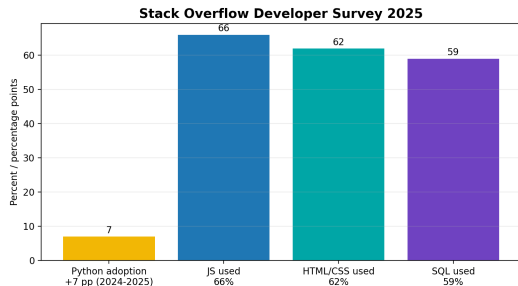
TIOBE Index, Aug 2026



PYPL Worldwide, Aug 2026



Sumber: TIOBE Index Aug 2026; PYPL Worldwide Aug 2026. Angka dapat berubah, tetapi pola dominasi Python dalam ekosistem data dan komputasi masih kuat.



## Kekuatan praktis Python

- Sintaks relatif mudah dibaca.
- Ekosistem library kuat: NumPy, pandas, matplotlib, SciPy, scikit-learn.
- Cocok untuk prototyping, analisis data, model, dan automasi.
- Mendukung Jupyter/Google Colab untuk pembelajaran berbasis notebook.

Sumber: Stack Overflow Developer Survey 2025 dan dokumentasi ekosistem Python.

## Ilmu data

- Data cleaning dan wrangling.
- Statistik deskriptif dan visualisasi.
- Regresi, klasifikasi, clustering.
- Interpretasi hasil untuk keputusan.

## Scientific computing

- Aljabar linier komputasional.
- Simulasi numerik.
- Analisis algoritma dan eksperimen performa.
- Reproducible analysis.

**Nilai tambah Matematikawan:** tidak hanya memanggil library, tetapi memahami struktur model, asumsi, dan batas kesimpulan.

# Kontrak kuliah: yang ditekankan

## Fokus perkuliahan

- Kode Python yang benar, terbaca, dan dapat dijalankan ulang.
- Logika algoritmik: urutan langkah, kondisi, perulangan, dan testing.
- Data: membaca, merepresentasikan, membersihkan, dan menganalisis.
- Project akhir: program Python dan draft proposal PKM.

## Sikap belajar

- Aktif bertanya, mencoba, dan berbagi solusi.
- Tidak cukup menyalin kode; harus bisa menjelaskan baris penting.
- Kesalahan program adalah bahan belajar, bukan aib.

## Asesmen dan pembagian minggu

| Komponen                                           | Bobot | Minggu                       |
|----------------------------------------------------|-------|------------------------------|
| Praktikum                                          | 15%   | M1-M5, M7, M9-M12            |
| Tugas mandiri                                      | 15%   | M1-M5, M7, M9-M12            |
| Quiz e-learning                                    | 10%   | 10 quiz: M1-M5, M7, M9-M12   |
| UTS                                                | 25%   | M8                           |
| Final Project: Program Python + Draft Proposal PKM | 35%   | M12-M16: 5%, 5%, 7%, 8%, 10% |

### Project PKM

Satu kelompok tugas besar dapat dipisah menjadi dua kelompok saat pengajuan PKM, jika diperlukan untuk menyesuaikan aturan jumlah anggota dan kelayakan ide.

## BAP dan distribusi pengampu

| Pengampu             |            | Minggu             | Fokus                                                                                                      |
|----------------------|------------|--------------------|------------------------------------------------------------------------------------------------------------|
| Triyana S.Si., M.Si. | Muliawati, | M2, M3, M7, M9-M12 | Fungsi, NumPy, aljabar linier, regresi, kNN, k-Means, problem framing PKM                                  |
| Rifky Fauzi          |            | M1, M4-M6, M13-M16 | Konsep algoritma, rekursi, Big-O, searching/sorting, analisis kinerja, finalisasi program dan proposal PKM |
| Tim Pengampu         |            | M8                 | UTS                                                                                                        |

### Praktik dan asesmen mingguan

Praktik, tugas mandiri, dan quiz: M1-M5, M7, M9-M12. Project: M12-M16.

# Project akhir: program Python + draft proposal PKM

## Alur project

- M12: problem framing, data/mitra, kandidat skema PKM.
- M13: studi literatur, metode, dan outline proposal.
- M14: demo program Python versi awal dan reproducibility.
- M15: draft proposal PKM dan program hampir final.
- M16: presentasi final, program Python final, dan draft proposal PKM.

## Skema PKM

Skema tidak dikunci sejak awal. Dosen akan mengarahkan sesuai masalah, data, mitra, dan bentuk solusi.

## Referensi utama: tiga buku, tiga fungsi

| Rujukan               | Peran dalam MK                                                                                                                   | Minggu dominan |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------|----------------|
| Miller & Ranum (2013) | Fondasi IndoMS: problem solving, algoritma, data, variabel, control structures, functions, Big-O, recursion, searching, sorting. | M1-M6          |
| Pine (2019)           | Scientific computing: NumPy array, vektor/matriks, komputasi numerik, dan aljabar linier komputasional.                          | M3, M7         |
| McKinney (2022)       | Data analysis/data science: NumPy, pandas, wrangling, visualisasi, workflow data, dan pengantar modeling.                        | M9-M12         |

### Keputusan kurikulum

Pine tidak diganti. Miller & Ranum melengkapi bagian algoritma dan Big-O; Pine menjaga identitas scientific computing; McKinney menjaga arah data science.

## Blok komputasi M1-M7: materi dan rujukan

| M | Materi inti                                                                             | Rujukan utama            |
|---|-----------------------------------------------------------------------------------------|--------------------------|
| 1 | Konsep algoritma dan pemrograman; data, variabel, pernyataan, operasi, input/output     | Miller Ch. 1             |
| 2 | Alur logika: sequential, selection, iteration; fungsi dan prosedur secara konseptual    | Miller 1.4.3; 1.4.5      |
| 3 | Representasi algoritma: narasi, pseudocode, flowchart; list/array dan pengantar NumPy   | Miller 1.4.1; Pine NumPy |
| 4 | Subprogram, rekursi, base case, recursive case, tracing, dan pengujian sederhana        | Miller Ch. 4             |
| 5 | Analisis algoritma: $T(n)$ , dominant term, Big-O, dan time-space trade-off             | Miller Ch. 2             |
| 6 | Searching, sorting, dan analisis kinerja algoritma dengan benchmark <code>timeit</code> | Miller 2.3; Ch. 5        |
| 7 | Larik multidimensi, vektor, matriks, sistem linear, dan solver komputasional            | Pine 9.3                 |

### Catatan IndoMS

Istilah IndoMS dibuat eksplisit pada SAP: konsep algoritma, representasi algoritma, data/variabel, pernyataan/operasi, alur logika, larik, subprogram, rekursi, dan kompleksitas algoritma.

# Materi Pertemuan 1: Python dasar

## Yang dipelajari

- Variabel dan ekspresi.
- Input dan output.
- Tipe data `int` dan `float`.
- Operasi aritmatika.
- Kondisional `if`.
- Perulangan `for` dan `while`.

## Target minimum

Mahasiswa dapat membuat program/notebook Python yang menerima input sederhana, memproses data numerik, memilih cabang keputusan, dan mengulang proses dengan loop.

# Menjalankan Python: pilih lingkungan kerja yang nyaman

## Pilihan lingkungan kerja

- Google Colab: praktis melalui browser, tanpa instalasi lokal.
- Jupyter Notebook/Lab: cocok untuk komputasi ilmiah dan eksplorasi data.
- IDE/editor lokal: VS Code, Spyder, PyCharm, atau editor lain yang mendukung Python.

```
print("Halo, Matematika ITERA!")  
print(2 + 3)
```

```
nama = "Alya"  
semester = 3  
ipk = 3.45  
  
print(nama)  
print(semester)  
print(ipk)
```

## Model mental

- Variabel adalah nama yang merujuk ke suatu objek/nilai.
- Operator = berarti assignment, bukan sama dengan matematis.
- `print()` menampilkan nilai ke layar.

```
nama = input("Nama: ")
umur = int(input("Umur: "))

print("Halo", nama)
print("Tahun depan umurmu", umur + 1)
```

## Catatan penting

- Hasil `input()` selalu berupa string.
- Gunakan `int()` atau `float()` jika perlu angka.
- Output harus membantu manusia memahami hasil program.

# Bilangan bulat di komputer: dari desimal ke biner

Contoh:  $13_{10}$

$$13 = 8 + 4 + 1 = 2^3 + 2^2 + 2^0$$

$$13_{10} = 1101_2$$

Dalam 8 bit unsigned:

00001101<sub>2</sub>

Bobot setiap bit

| bit   | 7   | 6  | 5  | 4  | 3 | 2 | 1 | 0 |
|-------|-----|----|----|----|---|---|---|---|
| bobot | 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 |
| nilai | 0   | 0  | 0  | 0  | 1 | 1 | 0 | 1 |

Mengapa jumlah bit penting?

8-bit unsigned hanya punya  $2^8 = 256$  pola: 0 sampai 255. Dengan signed two's-complement: -128 sampai 127.

## Tipe data: int dan float

```
a = 7 # int
b = 7.0 # float
c = 2 ** 100 # int besar di Python

print(type(a))
print(type(b))
print(c)
```

### Inti konsep

- int: bilangan bulat.
- float: bilangan riil aproksimasi di komputer.
- Python int dapat sangat besar selama memori cukup.
- Di level mesin/NumPy, batas 32-bit dan 64-bit tetap penting.

## Batas bilangan pada komputer

| Representasi | Contoh rentang                      | Makna                                           |
|--------------|-------------------------------------|-------------------------------------------------|
| int32 signed | $-2^{31}$ sampai $2^{31} - 1$       | sekitar $-2,147,483,648$ sampai $2,147,483,647$ |
| uint32       | 0 sampai $2^{32} - 1$               | sekitar 0 sampai $4,294,967,295$                |
| int64 signed | $-2^{63}$ sampai $2^{63} - 1$       | sekitar $\pm 9.22 \times 10^{18}$               |
| float32      | maks. sekitar $3.4 \times 10^{38}$  | presisi sekitar 7 digit desimal                 |
| float64      | maks. sekitar $1.8 \times 10^{308}$ | presisi sekitar 15-17 digit desimal             |

**Pelajaran:**

komputer menghitung dengan representasi terbatas. Matematika ideal, komputer aproksimatif.

## Dari 5.75 ke representasi floating-point

### Langkah biner

$$5.75_{10} = 101.11_2 = 1.0111_2 \times 2^2$$

Untuk IEEE-754 float32:

- sign: 0
- exponent:  $2 + 127 = 129 = 10000001_2$
- fraction: 0111000...

|   |  |          |  |            |
|---|--|----------|--|------------|
| 0 |  | 10000001 |  | 0111000... |
|---|--|----------|--|------------|

```
x = 5.75
print(x)
print(0.1 + 0.2)
print((0.1 + 0.2) == 0.3)
```

### Pesan

Sebagian pecahan desimal tidak punya representasi biner hingga. Karena itu float adalah aproksimasi, bukan bilangan real matematis yang persis.

## Perangkat bit kecil: fixed-point sebagai alternatif

### Contoh pedagogis Q8.8 (16 bit)

Simpan 8 bit untuk bagian bulat dan 8 bit untuk pecahan.

$$5.75 \times 2^8 = 1472$$

$$1472_{10} = 00000101\ 11000000_2$$

Saat dibaca kembali:

$$1472/256 = 5.75$$

### Mengapa relevan dengan komputer lama?

Pada banyak perangkat 8/16-bit dan game console generasi lama, tidak tersedia unit floating-point cepat. Programmer sering memilih integer atau fixed-point agar komputasi lebih ringan dan perilakunya terkontrol.

### Catatan

Q8.8 di sini adalah contoh format fixed-point untuk memahami ide representasi; bukan format wajib suatu konsol tertentu.

```
a = 10
b = 3

print(a + b) # tambah
print(a - b) # kurang
print(a * b) # kali
print(a / b) # bagi biasa
print(a // b) # bagi bulat
print(a % b) # sisa bagi
print(a ** b) # pangkat
```

## Prioritas operasi

Urutan umum mengikuti PEMDAS:

- Parentheses.
- Exponent.
- Multiplication / Division.
- Addition / Subtraction.

# Kondisional if

```
nilai = float(input("Nilai: "))

if nilai >= 75:
    status = "Lulus"
elif nilai >= 60:
    status = "Remedial"
else:
    status = "Belum lulus"

print(status)
```

## Pola berpikir

- Komputer memilih cabang berdasarkan kondisi logis.
- Kondisi bernilai True atau False.
- Indentasi menentukan blok program.

# Perulangan for dan while

```
# for: jumlah pengulangan jelas
for i in range(5):
    print(i)
```

```
# while: ulang selama kondisi benar
n = 1
while n < 100:
    n = 2 * n
print(n)
```

## Hubungan dengan Matematika

Perulangan mirip proses iteratif: barisan, metode numerik, simulasi, dan optimisasi semuanya memerlukan ide pengulangan.

## Sebelum kerja kelompok: hal yang biasanya tidak disukai

### Masalah klasik

- Penumpang gratis.
- Diam saat diskusi, muncul saat nilai keluar.
- Deadline mepet karena tidak ada progres mingguan.
- File tercecer dan versi program membingungkan.
- Copy-paste kode tanpa paham.

### Kesepakatan project

- Ada pembagian tugas dan log kontribusi.
- Setiap anggota wajib bisa menjelaskan bagian penting.
- Progress M12-M16 harus menunjukkan capaian dan kendala.
- Kode harus dapat dijalankan ulang pada lingkungan Python yang disepakati kelompok.

## Kerjakan mandiri menggunakan Python

1. Buat file/notebook dengan nama: Minggu01\_PythonDasar\_Nama.
2. Tulis 5 variabel dengan tipe data berbeda.
3. Buat program menghitung akar persamaan kuadrat sederhana.
4. Buat program menghitung jarak dua titik.
5. Tambahkan minimal 5 komentar yang menjelaskan kode.
6. Kumpulkan file atau tautan sesuai instruksi e-learning.

Lingkungan kerja bebas: Google Colab, Jupyter, Spyder, VS Code, atau IDE Python lain.

- Bradley N. Miller dan David L. Ranum. *Problem Solving with Algorithms and Data Structures Using Python*. Release 3.0, 2013.
- David J. Pine. *Introduction to Python for Science and Engineering*. CRC Press, 2019.
- Wes McKinney. *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter*. 3rd ed. O'Reilly Media, 2022.
- Python Documentation: <https://docs.python.org/3/>
- NumPy Documentation: <https://numpy.org/doc/>
- pandas Documentation: <https://pandas.pydata.org/docs/>
- TIOBE Index Aug 2026; PYPL Worldwide Aug 2026; Stack Overflow Developer Survey 2025.